TamNet^{Club} – Architecture Deep Dive

Overview

TamNet^{Club} is a story-centric, verdict-driven social platform built around structured participation, explicit moderation, and reputation-based progression.
Unlike traditional social systems that prioritise open-ended interaction, TamNet^{Club} enforces constrained participation, measurable outcomes, and accountable behaviour.
The architecture is product-driven, with engineering decisions directly reflecting core constraints such as gated membership, verdict-based interaction, anti-abuse controls, and behaviour-driven rank progression.

1. High-Level Architecture

TamNet^{Club} follows a layered full-stack architecture:

Client → API → Services → Repositories → Database

Stack

- Frontend: React 19, Vite, React Router
- Backend: Spring Boot, Spring Security, REST API
- Database: Relational model with Flyway-managed migrations
- Security: Spring Security + custom session filter
- Analytics: PostHog

Architectural Responsibilities

Frontend

- handles routing and protected views
- manages onboarding, reentry UX, voting, profile views, reporting flows
- renders admin moderation interfaces
- communicates via cookie-authenticated HTTP requests

Backend API

- owns all domain logic
- handles:
    - membership lifecycle
    - session issuance and recovery
    - case lifecycle
    - vote handling and final verdict resolution
    - ranking computation and penalties
    - admin moderation actions

Service Layer

- centralises business rules
- ensures consistent enforcement of:
- vote validation
- ranking updates
- moderation effects

Repository Layer

- abstracts database access
- provides domain-specific queries
- supports efficient retrieval for:
- members
- cases
- votes
- moderation state

2. Repository Structure

Frontend (React + Vite)

```
tamnet-frontend/
├── src/
│   ├── assets/              # badges and visual assets
│   ├── components/          # shared UI and modal components
│   ├── constants/           # session and client constants
│   ├── features/cases/      # case-specific client helpers
│   ├── hooks/               # route/auth/member/admin state hooks
│   ├── pages/               # route-level screens
│   │   └── admin/           # admin dashboard pages
│   ├── services/            # API client and analytics integration
│   ├── test/                # frontend test setup
│   └── utils/               # formatting and presentation helpers
```

Backend (Spring Boot)

```
tamnetclub/
├── src/main/java/com/tamnetclub/
│   ├── config/              # Spring, Flyway, and security config
│   ├── controller/          # member, case, vote, admin endpoints
│   ├── dto/                 # API response/request shapes
│   ├── exception/           # centralized exception handling
│   ├── model/               # domain entities and enums
│   ├── repository/          # data access layer
│   ├── security/            # custom filters and rate limiting
│   └── service/             # business logic and ranking engine
└── src/main/resources/
    ├── application.properties
    ├── application-dev.properties
    ├── db/migration/        # Flyway schema evolution
    ├── static/
    └── templates/
```

Design Benefits

- clear separation of concerns
- scalable backend layering
- maintainable domain structure
- versioned database evolution

3. System Architecture

Architecture Diagram

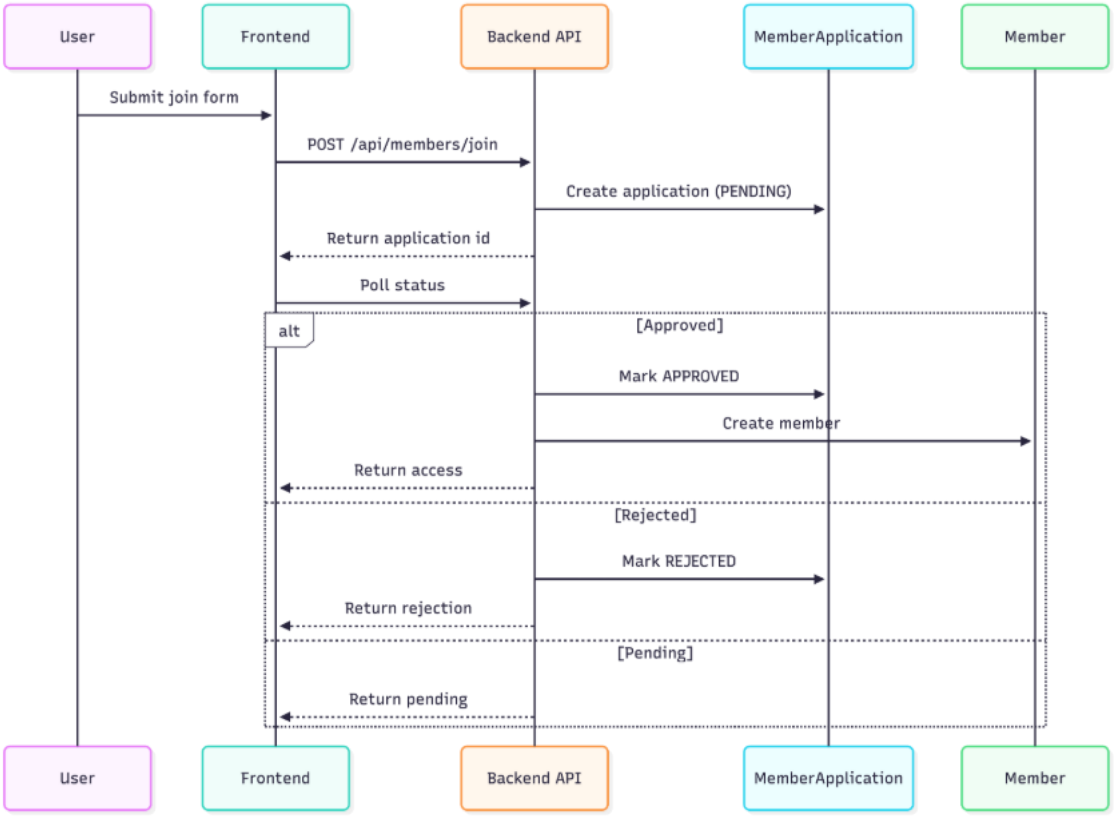
System Architecture — Request flow from client through security pipeline to domain services and persistence layer.

Key Design Insight
Security is part of the request lifecycle, not a separate layer.
This ensures:

- consistent enforcement
- reduced attack surface
- predictable behaviour

4. Core Flows

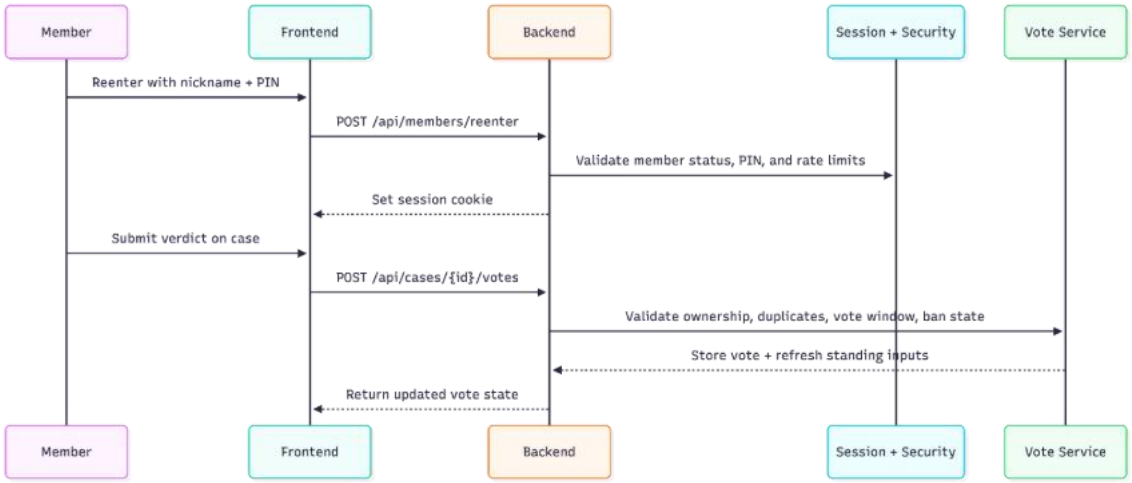
Join / Approval Flow



Join Flow – Application lifecycle including pending, approved, and rejected states.

Design Insight:
Separating MemberApplication from Member prevents polluted data and simplifies approval workflows.

Reentry and Voting Flow



Reentry and Voting Flow – Secure member session restoration followed by validated interaction processing, ensuring integrity, access control, and consistent case state evolution.

Resolution / Ranking / Moderation Flow
Case Resolution Flow

- votes aggregated
- NOT_ENOUGH_INFO excluded
- final verdict computed
- result feeds ranking system

Admin Moderation Flow

- admin views reports and metrics
- performs actions:
  - hide/unhide case
  - ban/unban member
- updates persist immediately

5. Authentication & Security Design

TamNet^{Club} uses dual authentication models.
Member Auth

- member requests are authenticated through a dedicated session cookie
- the cookie is `HttpOnly`
- production requests use `Secure`
- cookie scope uses `SameSite=Lax`
- a custom member-session filter resolves the cookie into member authentication for protected API routes

Admin Auth

- admin login is handled through Spring Security
- admin endpoints require `ROLE_ADMIN`
- admin sessions are intentionally separate from member sessions

Security Controls
CSRF:

- enabled for state-changing member actions such as logout, case submission, voting, reporting, and profile updates

CORS:

- restricted origins only; wildcard origins with credentials are not allowed

IP-based throttling / rate limiting:

- applied to:
  - join
  - login/reentry
  - voting
  - reporting
  - case
  - submission
  - admin login

Session validation:

Requests are rejected if:

- session expired
- member banned
- member invalid

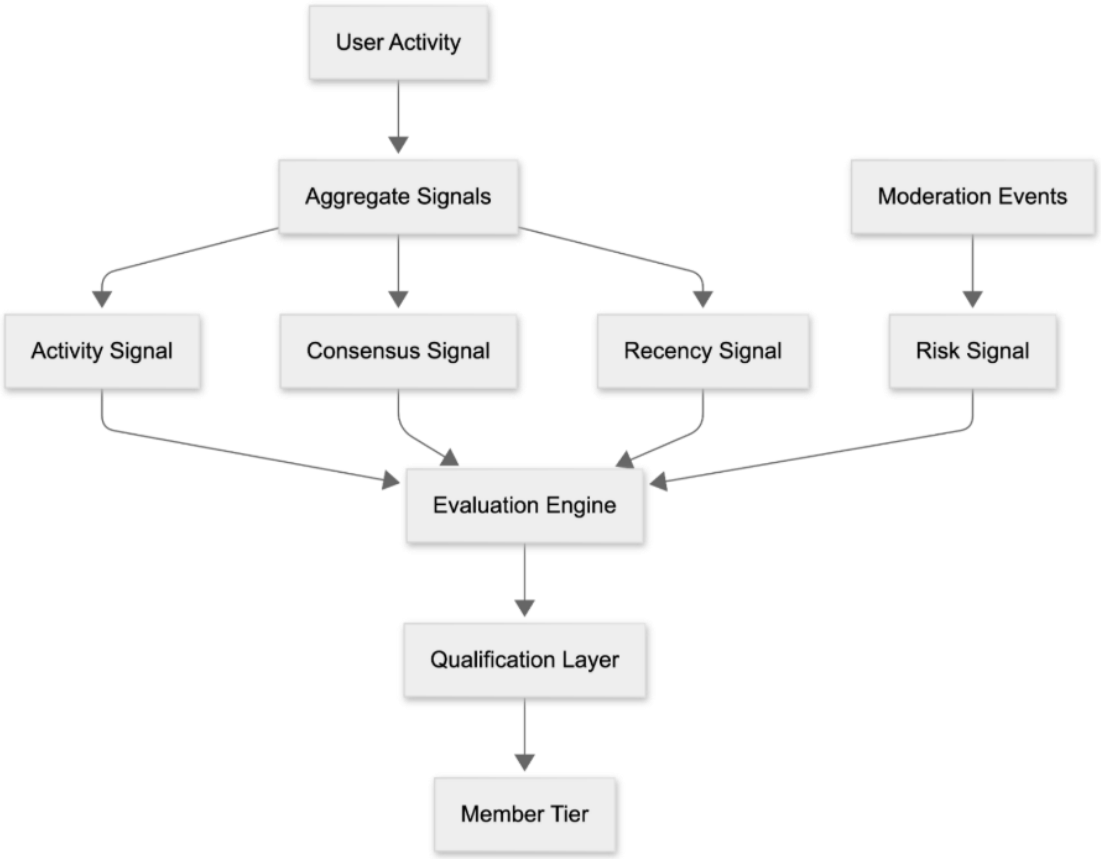
The backend uses IP-keyed request throttling at the filter layer for sensitive endpoints, with optional forwarded-header support behind a trusted proxy. Reentry and recovery flows also use persistent IP-based attempt tracking so repeated failures can trigger cooldowns even outside the short in-memory filter window.

Recovery Model

Layered recovery model:

- nickname + PIN (primary)
- member code (backup)
- recovery phrase (reset)

6. Ranking System Design



Ranking System – Multi-signal computation combining participation, accuracy, recency, and penalties before applying rank thresholds. The ranking system is multi-dimensional and behaviour-driven.

Ranking Signals

- activity signal
- consensus signal
- recency signal
- risk signal

Promotion Constraints

- minimum participation thresholds
- minimum consistency and accuracy
- recent activity requirements
- acceptable moderation history

System Properties

- contributions are capped to prevent farming
- activity decays over time
- penalties reduce progression
- volume alone does not determine rank

Neutral Signal Handling

NOT_ENOUGH_INFO:

- stored as vote
- excluded from resolution
- it does not affect accuracy
- it does not count toward resolved vote or accuracy scoring
- it contributes only a small participation-only credit

This preserves system integrity while allowing the club to express uncertainty.

7. Ranking Precision & Progress

The ranking engine uses a decimal-based scoring model, ensuring that all contributions are accumulated with full precision. Partial signals (such as limited-information interactions) are included as small participation credits without affecting accuracy or resolution.

Rank eligibility is evaluated using exact score comparisons, with no rounding applied before threshold checks. This allows continuous score progression while maintaining clear, discrete promotion thresholds.
A clear separation is maintained between:

- internal scoring – precise, continuous values used for all ranking logic
- display metrics – simplified values used for user-facing feedback

Progress toward the next rank is calculated as a percentage derived from the current score and the next threshold. This percentage is intentionally rounded for clarity in the UI, while the underlying ranking calculations remain unaffected.

8. Data Model Overview

Core entities include:

- Member
- MemberApplication
- TamnetCase
- TamnetVote
- MemberRecoveryChallenge
- Join / reentry attempt entities

Key Design Choice
Ownership centres on internal member IDs rather than user-facing credentials.
Benefits:

- stronger data integrity
- maintainability
- future scalability

9. Admin System

The admin layer is intentionally moderation-focused rather than a broad internal operations platform.
Admin Capabilities

- review members
- ban / unban members
- review all cases, including hidden ones
- hide / unhide cases
- inspect reported content
- view high-level moderation metrics

10. Product Decisions & Trade-offs

No Comments Initially

- reduces moderation complexity
- keeps interaction constrained and analysable

Verdict-Based Interaction

- easier to moderate
- easier to reason about
- directly compatible with ranking and consensus logic

72-Hour Voting Window

- encourages timely engagement
- simplifies resolution rules
- prevents endless re-litigation of older cases

Prestige Over Speed

The rank system is intentionally strict. The product favours earned credibility over fast progression.

Operational Trade-offs

Deliberately simplified for early-stage use:

- no distributed caching layer yet
- no asynchronous ranking job pipeline yet
- no large-scale admin operations platform yet

11. Scalability Outlook

Current Strengths

- layered architecture
- relational model with evolving migrations
- DTO-based API surface
- moderation and auth concerns separated from UI code

Future Improvements

- async ranking computation
- feed caching
- strengthen database indexing for high-volume moderation and standing queries
- introduce caching for profiles, metrics, and feed results

12. Testing Strategy

The system is validated through a combination of integration and service-level tests.

- integration tests cover join, approval, reentry, recovery, case submission, voting, and standing refresh flows
- ranking-focused tests cover score accumulation, threshold evaluation, penalty handling, and neutral-signal isolation
- security tests cover auth boundaries, session expiry, banned-member rejection, CSRF-protected actions, and separation between member and admin access

Key scenarios include:

- incremental participation accumulation, including precise decimal handling for small signals such as `NOT_ENOUGH_INFO`
- exclusion of neutral signals from accuracy, resolved-vote counts, and final verdict resolution
- member-only versus admin-only endpoint protection
- logout and session flows under CSRF enforcement

13. Deployment Readiness

- environment-based configuration
- Flyway migrations
- secure cookies in production
- HTTPS-aware request handling
- CSRF and CORS enforcement
- active rate limiting
- hosted database support
- analytics hooks through PostHog

Summary

TamNet^{Club} demonstrates:

- full-stack engineering across React, Spring Boot, and a relational data layer
- security awareness through session design, CSRF, CORS, rate limiting, and auth separation
- product thinking through constrained interaction design, moderation workflows, and gated membership

This project demonstrates the design and implementation of a product-oriented system, combining domain-driven logic, clear architectural boundaries, and security-conscious engineering to support real-world interaction patterns.